

# Linux Monitoring Stack

## Sprint Plan & Team Collaboration Guide

---

|           |                                                     |
|-----------|-----------------------------------------------------|
| Project   | Linux Infrastructure Monitoring Stack               |
| Team Size | 3 Interns                                           |
| Duration  | 2 Weeks (10 Working Days)                           |
| Stack     | Prometheus + Node Exporter + Alertmanager + Grafana |
| Version   | 1.0                                                 |
| Status    | Sprint Planning                                     |

## Table of Contents

---

|                                    |    |
|------------------------------------|----|
| 1. Project Overview                | 3  |
| 2. Team Roles & Responsibilities   | 4  |
| 3. Sprint Schedule — Week 1        | 6  |
| 4. Sprint Schedule — Week 2        | 8  |
| 5. Integration & Handoff Plan      | 10 |
| 6. Shared Git Repository Structure | 11 |
| 7. Alert Rules Reference           | 12 |
| 8. RACI Matrix                     | 14 |
| 9. Definition of Done              | 15 |
| 10. Daily Standup Protocol         | 16 |
| 11. Risk Register                  | 16 |

# 1. Project Overview

This document serves as the complete planning and execution guide for a two-week sprint to implement a production-ready Linux server monitoring stack. The stack consists of four open-source components — Prometheus, Node Exporter, Alertmanager, and Grafana — and will be built and delivered by a team of three interns working in parallel across clearly separated ownership boundaries.

The sprint is organized so that each person owns one architectural layer of the stack end-to-end, from installation through validation. This minimizes inter-dependencies during the first week and converges on three defined integration points in the second week. The document covers the day-by-day task breakdown, integration handoffs, shared repository structure, all 20 alert rules, RACI accountability, and the definition of done.

## 1.1 Objectives

- Deploy Node Exporter on all target Linux servers and confirm metrics collection is live.
- Configure Prometheus with all scrape targets and 20 production-grade alert rules across 6 categories.
- Configure Alertmanager with email notification routing, grouping, and deduplication.
- Build Grafana dashboards providing a live visual overview of all monitored hosts.
- Validate the complete end-to-end pipeline: metric collection → alert evaluation → notification delivery → dashboard display.
- Commit all configuration files to a shared Git repository as the single source of truth.

## 1.2 Stack Architecture Summary

The stack follows a pull-based architecture. Node Exporter runs on each monitored server and exposes metrics on port 9100. Prometheus scrapes those endpoints every 15 seconds and stores data as time-series records. Alert rules are evaluated against live data; when a threshold is breached, Prometheus forwards the alert to Alertmanager, which routes and deduplicates notifications before delivering them by email. Grafana connects to Prometheus as a read-only data source and renders dashboards independently.

| Component     | Port | Runs On                | Owner    |
|---------------|------|------------------------|----------|
| Node Exporter | 9100 | Every monitored server | Person A |
| Prometheus    | 9090 | Monitoring VM          | Person A |
| Alertmanager  | 9093 | Monitoring VM          | Person B |
| Grafana       | 3000 | Monitoring VM          | Person C |

## 2. Team Roles & Responsibilities

---

The work is split by architectural layer. Each person owns one layer completely — installation, configuration, testing, and documentation of their component. This avoids merge conflicts, creates clear accountability, and makes it easy to identify who to consult when something breaks.

### Person A — Infrastructure & Data Collection

Primary components: Node Exporter (all target servers), Prometheus (monitoring VM), prometheus.yml configuration, scrape target management, self-monitoring alert rules.

#### Week 1 focus:

- Inventory all target servers and document IPs, hostnames, and OS versions.
- Open required firewall ports (9090, 9100, 9093, 3000) on all machines.
- Deploy and start Node Exporter as a systemd service on every target server.
- Install Prometheus on the monitoring VM and write prometheus.yml with all scrape targets.
- Verify all targets show as UP in the Prometheus Targets UI at :9090.

#### Week 2 focus:

- Load alert rule files from Person B and validate with promtool check rules.
- Reload Prometheus after each rule addition and confirm zero syntax errors.
- Write the four self-monitoring rules (Section 6.6) and load them.
- Set data retention to 15 days and document storage configuration.
- Act as gatekeeper: run promtool check config before any prometheus.yml change is merged.

Key deliverable handed to team: Prometheus instance at a known IP, all targets green, data flowing, Prometheus data source URL documented.

### Person B — Alerting Pipeline

Primary components: All alert rule YAML files (20 rules across 5 categories), Alertmanager installation, alertmanager.yml configuration, notification routing, end-to-end alert testing, runbooks.

#### Week 1 focus:

- Write all alert rule YAML files (cpu.yml, memory.yml, disk.yml, network.yml, system.yml).
- Validate every rule file offline with: promtool check rules rules/\*.yml

- Rules can be written before Prometheus is running — only the label values from Person A's handoff (Day 3) are needed to finalize the instance selectors.
- Coordinate with Person A on Day 3 to confirm instance label format used in `prometheus.yml`.
- Review PromQL expressions for all 20 rules against the Node Exporter metric name reference.

### Week 2 focus:

- Install Alertmanager on the monitoring VM and register as a systemd service.
- Write `alertmanager.yml`: email receiver, routing tree, `group_wait 30s`, `group_interval 5m`, `repeat_interval 4h`.
- Wire Prometheus to Alertmanager via the alerting: block in `prometheus.yml`.
- Execute end-to-end alert fire tests: temporarily lower thresholds, confirm email delivery.
- Write a one-page runbook for each critical severity alert: meaning, first steps, escalation.

Key deliverable handed to team: All 20 rules active with zero errors, confirmed email notification on at least 3 test fires, runbook document shared with Person C.

## Person C — Visualization & Documentation

Primary components: Grafana installation, all dashboards, dashboard export, final integrated documentation, and the Day 10 smoke test coordination.

### Week 1 focus:

- Study the Node Exporter Full dashboard (Grafana ID 1860) and plan panel layout for the team's specific hosts.
- Draft the architecture diagram for inclusion in the final documentation.
- Update the monitoring stack documentation with the expanded alert definitions from this sprint.
- Prepare the Grafana data source connection config (Prometheus URL, no auth) so Day 8 setup is instant.
- Maintain the shared Git repository: enforce commit message conventions, review PRs.

### Week 2 focus:

- Install Grafana on Day 8 (after Prometheus has 48+ hours of real data).
- Import Node Exporter Full dashboard (ID 1860) and customize panel titles and instance filters.
- Build an overview dashboard: one row per monitored server, stat panels for CPU, memory, disk, uptime.
- Export all dashboards as JSON and commit to the repo under `dashboards/`.
- Run the Day 10 final smoke test and compile the complete sprint retrospective document.

Key deliverable: Live Grafana at port 3000, dashboards showing all hosts in real time, all configs committed to the repo, final documentation updated.

### 3. Sprint Schedule — Week 1

Week 1 establishes the data collection foundation. All three workstreams run in parallel with a single synchronization point on Day 3.

| Day   | Owner     | Task                                                                                                                                                                                                                                                                                                                     | Deliverable                                                                                                                                     |
|-------|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Day 1 | A + B + C | Kick-off: create shared Git repo, agree on branch naming convention (feat/person-a, feat/person-b, feat/person-c), set up shared docs folder. Person A inventories all target servers and documents IPs/OS versions. Person B pulls the Node Exporter metric reference doc. Person C creates the documentation skeleton. | Shared repo live, server inventory doc committed, all team members have SSH access to monitoring VM.                                            |
| Day 1 | A         | Verify network reachability between monitoring VM and all target servers. Open firewall ports 9090, 9100, 9093, 3000. Document any blocked ports.                                                                                                                                                                        | Firewall config documented and committed.                                                                                                       |
| Day 2 | A         | Download Node Exporter binary (latest stable release from GitHub) on each target server. Write and enable systemd service unit file. Verify metrics endpoint: <code>curl http://&lt;host&gt;:9100/metrics   grep node_cpu_seconds_total</code>                                                                           | Node Exporter running as systemd service on all target servers, endpoints returning data.                                                       |
| Day 2 | B         | Write <code>cpu.yml</code> alert rules: HighCPUWarning (>80%, 2m), HighCPUCritical (>95%, 5m), HighIOWait (>20%, 5m), HighLoadAverage (>core count, 5m). Run: <code>promtool check rules cpu.yml</code>                                                                                                                  | <code>cpu.yml</code> validated with zero errors, committed to feat/person-b branch.                                                             |
| Day 2 | C         | Draft architecture diagram. Update monitoring stack documentation cover and introduction to reflect 3-person team structure. Set up repo folder structure.                                                                                                                                                               | Architecture diagram draft committed, documentation skeleton committed.                                                                         |
| Day 3 | A         | Install Prometheus on monitoring VM. Write <code>prometheus.yml</code> with all scrape targets (one job per host group, 15s scrape interval). Register as systemd service and start.                                                                                                                                     | Prometheus running, all targets showing UP in <code>:9090/targets</code> . Confirm with: <code>curl http://localhost:9090/api/v1/targets</code> |
| Day 3 | A → B, C  | HANDOFF 1: Person A shares the finalized instance label format and list of all scrape targets. Person B uses these to finalize instance selectors in all rule files. Person C uses them to name Grafana panels correctly.                                                                                                | Targets list committed to <code>docs/targets.txt</code> . Person B and C acknowledge receipt.                                                   |
| Day 3 | B         | Write <code>memory.yml</code> : LowMemoryWarning (<15%, 2m), LowMemoryCritical (<5%, 1m),                                                                                                                                                                                                                                | <code>memory.yml</code> validated, committed.                                                                                                   |

|       |       |                                                                                                                                                                                                                                                                     |                                                                                |
|-------|-------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
|       |       | SwapUsageHigh (>80%, 2m), OOMKillDetected (instant). Validate with promtool.                                                                                                                                                                                        |                                                                                |
| Day 4 | A     | Fine-tune Prometheus scrape config: add relabeling to attach environment and team labels to all targets. Test PromQL queries in the expression browser to confirm label structure matches what Person B's rules expect.                                             | Labels verified, prometheus.yml updated and reloaded.                          |
| Day 4 | B     | Write disk.yml: LowDiskWarning (>80%, 5m), LowDiskCritical (>95%, 5m), DiskFillingFast (predict_linear 4h), InodeExhaustionWarning (>80%, 5m), HighDiskReadLatency (>100ms, 2m), HighDiskWriteLatency (>100ms, 2m). Validate.                                       | disk.yml validated, committed.                                                 |
| Day 4 | C     | Study Node Exporter Full dashboard (ID 1860). Identify which panels are relevant, which need renaming, and which need custom PromQL for multi-host views. Write panel design notes.                                                                                 | Panel design notes committed to docs/grafana-plan.md.                          |
| Day 5 | B     | Write network.yml: HighRXBandwidth (>800Mbps, 2m), HighTXBandwidth (>800Mbps, 2m), HighNetworkErrorRate (>10/s, 2m), NetworkInterfaceDown (instant). Write system.yml: InstanceDown, UnexpectedReboot, HighZombieProcesses, HighFileDescriptors, ClockSkewDetected. | network.yml and system.yml validated and committed.                            |
| Day 5 | B → A | HANDOFF 2 (prep): Person B opens a PR with all 5 rule files. Person A reviews label selectors against the live prometheus.yml config. Both are online together for 30 minutes to resolve any mismatches before merge.                                               | PR reviewed and merged. Rule files in /etc/prometheus/rules/ on monitoring VM. |
| Day 5 | A     | Load all 5 rule files into Prometheus. Reload Prometheus. Confirm all rules appear in :9090/alerts with no syntax errors. Screenshot the Alerts tab for the sprint report.                                                                                          | All rules loaded, zero errors confirmed. Screenshot committed to docs/.        |
| Day 5 | C     | Week 1 review: update sprint log with what was completed, what is pending, any blockers. Ensure all branches are merged to main. Confirm repo state matches what is running on the monitoring VM.                                                                   | Sprint week-1 review doc committed.                                            |



## 4. Sprint Schedule — Week 2

Week 2 completes the alerting pipeline, adds visualization, and validates the full stack end-to-end. The second major integration point occurs on Day 8 when Grafana connects to Prometheus.

| Day   | Owner | Task                                                                                                                                                                                                                                                                                                                         | Deliverable                                                                                     |
|-------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Day 6 | B     | Install Alertmanager on monitoring VM. Write alertmanager.yml: email receiver (SMTP config), routing tree (critical routes immediately, warnings batched), group_wait: 30s, group_interval: 5m, repeat_interval: 4h. Register as systemd service.                                                                            | Alertmanager running, config validated with: amtool check-config alertmanager.yml               |
| Day 6 | A     | Update prometheus.yml alerting: block to point to Alertmanager at localhost:9093. Write the 4 self-monitoring rules (prometheus_self.yml): ConfigReloadFailure, ScrapeErrorsIncreasing, AlertmanagerNotReachable, TSDBCompactionFailures. Load and validate.                                                                 | Prometheus forwarding alerts to Alertmanager confirmed. Self-monitoring rules active.           |
| Day 6 | C     | Install Grafana on monitoring VM. Register as systemd service. Open port 3000. Log in and change default admin password. Add Prometheus as a data source (URL: http://localhost:9090).                                                                                                                                       | Grafana accessible at :3000, Prometheus data source showing green (Save & Test passes).         |
| Day 7 | B     | End-to-end alert fire test 1 (CPU): temporarily set HighCPUWarning threshold to 1%. Run: stress-ng --cpu 4 --timeout 180. Confirm alert fires in Prometheus, reaches Alertmanager, email arrives within 2 minutes. Restore threshold.                                                                                        | CPU alert test: PASS. Email delivery confirmed with timestamp.                                  |
| Day 7 | B     | End-to-end alert fire test 2 (Disk): create a large temp file to push a partition above 80%. Confirm LowDiskWarning fires. Remove file, confirm alert resolves. End-to-end alert fire test 3 (Instance Down): stop Node Exporter on one target, confirm InstanceDown fires within 2 minutes, restart and confirm resolution. | Disk and InstanceDown tests: PASS. All 3 test results documented in docs/alert-test-report.md.  |
| Day 7 | B     | Write runbook entries for all 6 critical-severity alerts: InstanceDown, LowMemoryCritical, LowDiskCritical, NetworkInterfaceDown, OOMKillDetected, AlertmanagerNotReachable. Each entry covers: meaning, first diagnostic commands, escalation path.                                                                         | Runbook committed to docs/runbook.md. Shared with Person C for cross-referencing in dashboards. |
| Day 7 | A     | Security hardening: configure nginx reverse proxy for Prometheus with basic auth if the monitoring VM is on a shared network. Set file permissions on all config files to 640. Document any credentials in a .env file (not committed to repo).                                                                              | Hardening steps documented in docs/security-notes.md.                                           |
| Day 8 | C     | HANDOFF 3: Grafana connects to live Prometheus data (now 48+ hours old). Import Node Exporter Full dashboard (ID 1860).                                                                                                                                                                                                      | Node Exporter Full dashboard live,                                                              |

|        |           |                                                                                                                                                                                                                                                                                                                 |                                                                                |
|--------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
|        |           | Customize: rename instance labels, set dashboard refresh to 30s, add team name and environment variables.                                                                                                                                                                                                       | showing data for all hosts.                                                    |
| Day 8  | C         | Build the Overview Dashboard: one row per monitored server, stat panels for CPU usage %, memory available %, root disk used %, uptime. Use colour thresholds matching alert severity levels (green / amber / red).                                                                                              | Overview dashboard live with one row per server, all panels showing real data. |
| Day 9  | C         | Export all Grafana dashboards as JSON. Commit to repo under dashboards/. Add Grafana version and Prometheus data source URL to the dashboard JSON provisioning config so dashboards can be restored from the repo without manual import.                                                                        | All dashboard JSONs committed. Grafana provisioning config committed.          |
| Day 9  | A + B + C | Documentation sprint: update the Linux Monitoring Stack Documentation with actual values — config file paths used, exact systemd unit file contents, Prometheus data source URL, Alertmanager SMTP config (redacted), actual scrape target list, screenshots of Prometheus Alerts tab and Grafana dashboards.   | Final documentation Word file updated and committed.                           |
| Day 10 | A + B + C | FINAL SMOKE TEST: Person A stops Node Exporter on one target server. Person B watches Prometheus Alerts tab and email inbox. Person C watches Grafana — the server row should go red. Confirm alert fires within 2 minutes and email arrives. Person A restarts Node Exporter and all three confirm resolution. | Smoke test: PASS. Screenshot of all three views during the test committed.     |
| Day 10 | C         | Sprint retrospective: document what was built, what worked, what was difficult, what would be done differently. Archive the final documentation. Tag the repo with v1.0.                                                                                                                                        | Retrospective doc committed. Repo tagged v1.0. Sprint COMPLETE.                |

## 5. Integration & Handoff Plan

There are three formal handoff points in the sprint where one person's output becomes the direct input for another person's work. Each handoff requires a 30-minute synchronous session (video call or in-person) to prevent label mismatches and config drift.

| When                 | From                | What                                                                                                                                                                                                                                            | To Whom / How                                                                                          |
|----------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Day 3<br/>EOD</b> | Person A            | Finalized scrape target list: hostname, IP, port, instance label as it appears in prometheus.yml. Format: plain text file committed to docs/targets.txt.                                                                                        | Person B (uses instance labels in PromQL selectors) + Person C (uses hostnames to name Grafana panels) |
| <b>Day 5<br/>EOD</b> | Person B → Person A | All 5 alert rule YAML files submitted as a pull request on the shared repo. Person A reviews label selectors against the live prometheus.yml config. Both online for 30 min to resolve any selector mismatches before merge.                    | Person A loads files into /etc/prometheus/rules/, reloads Prometheus, confirms zero errors.            |
| <b>Day 8<br/>SOD</b> | Person A → Person C | Prometheus data source URL, any auth credentials (via secure channel, not repo), confirmation that Prometheus has at least 48 hours of data. List of metric names used in Person B's alert rules for Person C to reference in dashboard panels. | Person C connects Grafana, imports dashboards, validates all panels show data.                         |

### 5.1 Daily Synchronization

Outside the three formal handoffs, a daily 10-minute standup keeps the team aligned. Each person answers three questions only: what was completed since yesterday, what is planned for today, and is anything blocked. The standup should not become a problem-solving session — blockers that need more than 2 minutes to resolve are taken offline between the relevant parties.

## 6. Shared Git Repository Structure

All configuration files, dashboards, and documentation live in a single Git repository. No team member works from local-only files. The main branch is always deployable; feature branches (feat/person-a, feat/person-b, feat/person-c) are merged to main only after passing promtool validation.

| Path / File                                               | Purpose                                                                                               |
|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <a href="#">prometheus/prometheus.yml</a>                 | Main Prometheus config: scrape targets, alerting block, rule file paths. Owner: Person A.             |
| <a href="#">prometheus/rules/cpu.yml</a>                  | CPU alert rules (4 rules). Owner: Person B.                                                           |
| <a href="#">prometheus/rules/memory.yml</a>               | Memory alert rules (4 rules). Owner: Person B.                                                        |
| <a href="#">prometheus/rules/disk.yml</a>                 | Disk space and I/O alert rules (6 rules). Owner: Person B.                                            |
| <a href="#">prometheus/rules/network.yml</a>              | Network alert rules (4 rules). Owner: Person B.                                                       |
| <a href="#">prometheus/rules/system.yml</a>               | System health alert rules (5 rules). Owner: Person B.                                                 |
| <a href="#">prometheus/rules/prometheus_self.yml</a>      | Prometheus self-monitoring rules (4 rules). Owner: Person A.                                          |
| <a href="#">alertmanager/alertmanager.yml</a>             | Alertmanager routing config and receivers. Owner: Person B. SMTP credentials in .env (not committed). |
| <a href="#">grafana/dashboards/overview.json</a>          | Exported JSON of the overview dashboard. Owner: Person C.                                             |
| <a href="#">grafana/dashboards/node_exporter.json</a>     | Exported JSON of the Node Exporter Full dashboard (customized). Owner: Person C.                      |
| <a href="#">grafana/provisioning/</a>                     | Grafana provisioning YAML so dashboards load automatically on fresh install. Owner: Person C.         |
| <a href="#">docs/targets.txt</a>                          | Scrape target inventory: hostname, IP, instance label. Owner: Person A. Updated Day 3.                |
| <a href="#">docs/alert-test-report.md</a>                 | Results of all end-to-end alert fire tests run on Days 7 and 10. Owner: Person B.                     |
| <a href="#">docs/runbook.md</a>                           | Runbook entries for all critical-severity alerts. Owner: Person B.                                    |
| <a href="#">docs/security-notes.md</a>                    | Nginx proxy config, file permission settings, auth approach. Owner: Person A.                         |
| <a href="#">docs/grafana-plan.md</a>                      | Dashboard panel design notes and panel-to-metric mapping. Owner: Person C.                            |
| <a href="#">docs/sprint-log.md</a>                        | Daily sprint log maintained by Person C. Updated end of each day.                                     |
| <a href="#">Linux_Monitoring_Stack_Documentation.docx</a> | Primary project documentation. Owner: Person C.                                                       |
| <a href="#">.env.example</a>                              | Template showing required environment variables (SMTP credentials, etc.) without actual values.       |

|                         |                                                    |
|-------------------------|----------------------------------------------------|
| <code>.gitignore</code> | Excludes .env, any credential files, OS artifacts. |
|-------------------------|----------------------------------------------------|

## 7. Alert Rules Reference

All 20 alert rules to be implemented by Person B, organized by category. Each rule includes the PromQL condition, severity level, and evaluation window. Rules must pass promtool check rules validation before being handed off to Person A for loading.

### 7.1 CPU Rules (cpu.yml)

| Alert Name      | Condition                                                                                                            | Severity | Window  |
|-----------------|----------------------------------------------------------------------------------------------------------------------|----------|---------|
| HighCPUWarning  | $(1 - \text{avg}(\text{rate}(\text{node\_cpu\_seconds\_total}\{\text{mode}=\text{'idle'}\}[5\text{m}]))) * 100 > 80$ | warning  | for: 2m |
| HighCPUCritical | same expression > 95                                                                                                 | critical | for: 5m |
| HighIOWait      | $\text{avg}(\text{rate}(\text{node\_cpu\_seconds\_total}\{\text{mode}=\text{'iowait'}\}[5\text{m}]))) * 100 > 20$    | warning  | for: 5m |
| HighLoadAverage | $\text{node\_load15} / \text{count}(\text{node\_cpu\_seconds\_total}\{\text{mode}=\text{'idle'}\}) > 1$              | warning  | for: 5m |

### 7.2 Memory Rules (memory.yml)

| Alert Name        | Condition                                                                                    | Severity | Window    |
|-------------------|----------------------------------------------------------------------------------------------|----------|-----------|
| LowMemoryWarning  | $\text{node\_memory\_MemAvailable\_bytes} / \text{node\_memory\_MemTotal\_bytes} * 100 < 15$ | warning  | for: 2m   |
| LowMemoryCritical | same expression < 5                                                                          | critical | for: 1m   |
| SwapUsageHigh     | $(\text{MemSwapTotal} - \text{MemSwapFree}) / \text{MemSwapTotal} * 100 > 80$                | warning  | for: 2m   |
| OOMKillDetected   | $\text{increase}(\text{node\_vmstat\_oom\_kill}[5\text{m}]) > 0$                             | critical | immediate |

### 7.3 Disk Rules (disk.yml)

| Alert Name             | Condition                                                                                             | Severity | Window   |
|------------------------|-------------------------------------------------------------------------------------------------------|----------|----------|
| LowDiskWarning         | $(\text{size} - \text{avail}) / \text{size} * 100 > 80$<br>{fstype!~'tmpfs devtmpfs'}                 | warning  | for: 5m  |
| LowDiskCritical        | same expression > 95                                                                                  | critical | for: 5m  |
| DiskFillingFast        | $\text{predict\_linear}(\text{node\_filesystem\_avail\_bytes}[1\text{h}], 4*3600) < 0$                | warning  | for: 10m |
| InodeExhaustionWarning | $(1 - \text{files\_free} / \text{files}) * 100 > 80$                                                  | warning  | for: 5m  |
| HighDiskReadLatency    | $\text{rate}(\text{read\_time}[5\text{m}]) / \text{rate}(\text{reads\_completed}[5\text{m}]) > 0.1$   | warning  | for: 2m  |
| HighDiskWriteLatency   | $\text{rate}(\text{write\_time}[5\text{m}]) / \text{rate}(\text{writes\_completed}[5\text{m}]) > 0.1$ | warning  | for: 2m  |

## 7.4 Network Rules (network.yml)

| Alert Name           | Condition                                                      | Severity | Window    |
|----------------------|----------------------------------------------------------------|----------|-----------|
| HighRXBandwidth      | rate(node_network_receive_bytes_total[5m]) * 8 > 800000000     | warning  | for: 2m   |
| HighTXBandwidth      | rate(node_network_transmit_bytes_total[5m]) * 8 > 800000000    | warning  | for: 2m   |
| HighNetworkErrorRate | rate(receive_errs[5m]) > 10 or<br>rate(transmit_errs[5m]) > 10 | warning  | for: 2m   |
| NetworkInterfaceDown | node_network_carrier{device!='lo'} == 0                        | critical | immediate |

## 7.5 System Health Rules (system.yml)

| Alert Name          | Condition                                                 | Severity | Window    |
|---------------------|-----------------------------------------------------------|----------|-----------|
| InstanceDown        | up == 0                                                   | critical | for: 2m   |
| UnexpectedReboot    | node_time_seconds -<br>node_boot_time_seconds < 600       | warning  | immediate |
| HighZombieProcesses | node_procs_zombie > 5                                     | warning  | for: 5m   |
| HighFileDescriptors | node_filefd_allocated /<br>node_filefd_maximum * 100 > 80 | warning  | for: 2m   |
| ClockSkewDetected   | abs(node_timex_offset_seconds)<br>> 0.05                  | warning  | for: 5m   |

## 7.6 Prometheus Self-Monitoring Rules (prometheus\_self.yml)

| Alert Name               | Condition                                                   | Severity | Window    |
|--------------------------|-------------------------------------------------------------|----------|-----------|
| ConfigReloadFailure      | prometheus_config_last_reload_successful == 0               | critical | immediate |
| ScrapeErrorsIncreasing   | increase(scrape_errors_total[5m]) > 0                       | warning  | for: 5m   |
| AlertmanagerNotReachable | prometheus_notifications_alertmanagers_discovered < 1       | critical | immediate |
| TSDBCompactionFailures   | increase(prometheus_tsdb_compactions_failed_total[15m]) > 0 | warning  | for: 15m  |

## 8. RACI Matrix

R = Responsible (does the work) A = Accountable (owns the outcome) C = Consulted (provides input) I = Informed (kept updated)

| Activity                                        | Person A | Person B | Person C |
|-------------------------------------------------|----------|----------|----------|
| Server inventory and firewall configuration     | R/A      | I        | I        |
| Node Exporter deployment on all target servers  | R/A      | I        | I        |
| Prometheus installation and prometheus.yml      | R/A      | C        | C        |
| Prometheus self-monitoring rules                | R/A      | C        | I        |
| promtool validation gatekeeper (all rule files) | A        | R        | I        |
| Alert rule YAML files (all 5 categories)        | C        | R/A      | I        |
| Alertmanager installation and config            | C        | R/A      | I        |
| End-to-end alert fire testing                   | C        | R/A      | I        |
| Alert runbooks                                  | I        | R/A      | C        |
| Grafana installation                            | I        | I        | R/A      |
| Dashboard import and customization              | I        | C        | R/A      |
| Dashboard JSON export and repo commit           | I        | I        | R/A      |
| Shared Git repository maintenance               | C        | C        | R/A      |
| Sprint log and daily updates                    | I        | I        | R/A      |
| Final documentation update                      | C        | C        | R/A      |
| Day 3 Handoff 1 (targets list)                  | R/A      | C        | C        |
| Day 5 Handoff 2 (rule files PR review)          | R        | A        | I        |
| Day 8 Handoff 3 (Prometheus data source info)   | R        | I        | A        |
| Day 10 final smoke test                         | R        | R        | R        |
| Sprint retrospective                            | C        | C        | R/A      |



## 9. Definition of Done

The sprint is considered complete only when every criterion in this table is verified. No partial credit. If any criterion fails on Day 10, the relevant owner has one additional day to resolve it before the sprint is formally closed.

| Criterion                                                  | Owner    | Verification                                                                                                                                             |
|------------------------------------------------------------|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| All target hosts show UP in Prometheus                     | Person A | Screenshot of :9090/targets — zero targets in DOWN or UNKNOWN state.                                                                                     |
| All 20 alert rules loaded with zero syntax errors          | Person A | Output of: promtool check rules rules/*.yml — no errors. Screenshot of :9090/alerts.                                                                     |
| promtool check config passes cleanly                       | Person A | Command output showing SUCCESS committed to docs/.                                                                                                       |
| Alertmanager running and reachable from Prometheus         | Person B | AlertmanagerNotReachable alert is in inactive state. :9093 is accessible.                                                                                |
| End-to-end CPU alert test passed                           | Person B | Email received within 2 minutes of stress test start. Timestamp in alert-test-report.md.                                                                 |
| End-to-end Disk alert test passed                          | Person B | Email received after creating large temp file. Documented in alert-test-report.md.                                                                       |
| End-to-end InstanceDown alert test passed                  | Person B | Email received within 2 minutes of stopping Node Exporter. Documented.                                                                                   |
| Alert grouping confirmed                                   | Person B | Multiple simultaneous alerts produce a single grouped notification, not separate emails.                                                                 |
| Runbooks written for all 6 critical alerts                 | Person B | docs/runbook.md exists with an entry for each critical-severity rule.                                                                                    |
| Grafana accessible at port 3000                            | Person C | URL reachable from team member's browser. Login works.                                                                                                   |
| Node Exporter Full dashboard live for all hosts            | Person C | All host instance selectors return data. No empty panels.                                                                                                |
| Overview dashboard showing all monitored servers           | Person C | One row per server, CPU / memory / disk / uptime panels all populated.                                                                                   |
| All Grafana dashboard JSONs committed to repo              | Person C | dashboards/ folder in repo. JSON files are valid Grafana export format.                                                                                  |
| All config files committed to repo (no local-only configs) | All      | git status on monitoring VM shows clean working tree for all config paths.                                                                               |
| Final documentation updated with real config values        | Person C | Docx file includes actual file paths, systemd unit contents, scrape target list, screenshots.                                                            |
| Day 10 smoke test: full pipeline verified end-to-end       | All      | All three team members witness: host goes down → Prometheus alert fires → email arrives → Grafana panel turns red → host recovers → everything resolves. |



## 10. Daily Standup Protocol

Time: 10 minutes, same time each working day. Format: each person answers three questions in under 2 minutes.

- **Q1.** What did I complete since yesterday?
- **Q2.** What will I work on today?
- **Q3.** Is anything blocking me that I cannot resolve alone?

Blockers needing more than 2 minutes to discuss are taken offline between the relevant two people immediately after standup. Person C logs a one-line summary of each standup in docs/sprint-log.md.

## 11. Risk Register

| Risk                                                     | Probability | Impact | Mitigation                                                                                                              |
|----------------------------------------------------------|-------------|--------|-------------------------------------------------------------------------------------------------------------------------|
| Firewall ports blocked on target servers                 | Medium      | High   | Person A verifies on Day 1. If blocked, escalate to supervisor immediately — this gates all subsequent work.            |
| Instance label mismatch between rules and prometheus.yml | Medium      | Medium | Handoff 1 on Day 3 explicitly syncs label format. Both Person A and B review selectors together before merge.           |
| SMTP relay not available for Alertmanager                | Low         | High   | Person B confirms SMTP availability with supervisor on Day 1. Webhook (Slack/Teams) is the backup channel.              |
| Prometheus scrape target unreachable intermittently      | Low         | Medium | Use node_exporter's --web.listen-address flag to bind to the correct interface. Document in security-notes.md.          |
| Grafana has insufficient data on Day 8                   | Low         | Low    | Person C begins Grafana on Day 8 only if Prometheus started by Day 3. 5 days of data is sufficient for all panels.      |
| Team member unavailable for a day                        | Medium      | Medium | Sprint log and repo state allow any team member to pick up any other person's work. Repo is the single source of truth. |